First-Order Masked Fine-Shuffling Implementation Against Side-Channel Attacks with Application to ML-KEM

Noura Ait Manssour[1], Souhayl Ben El Haj Soulami✉ [1,2][0009−0006−2085−9324], Sylvain Duquesne [2][0000−0002−3854−8253], and Guillaume Fumaroli[1][0009−0003−3310−0390]

[1] Valeo, Créteil, France, noura.ait-manssour@valeo.com, souhayl.ben-el-haj-soulami@valeo.com, , guillaume.fumaroli@valeo.com
[2] Univ Rennes, CNRS, IRMAR - UMR 6625, F-35000 Rennes, France, sylvain.duquesne@univ-rennes.fr

Abstract. In 2020, Ravi *et al.* [23] published three shuffling variants with each offering a different performance-security trade-off for protecting the Numeric theoretic Transform (NTT). Among them, the *fine-shuffling* was proposed as the lightweight variant. The idea is to randomise the order of loading and storing the operands of the butterfly computation using conditional swapping based on random control bit. However, as noted by the authors themselves, *basic-fine-shuffling* implementation suffered from an obvious attack on the conditional swapping mask. So they introduced the *bitwise-fine-shuffling* to fix this issue. In this paper, we break this implementation using a template attack. The idea is to aggregate the leakage from the 16 bitwise `AND` operations used to construct the swap mask. The attack has been performed both on simulation and on a `STM32F303` target. Then, we propose a *masked fine-shuffling* variant to protect the *fine-shuffling* operation. The idea is to mask the secret-dependent memory accesses and apply the swap mask over its boolean sharing instead. The implementation is proven secure against first-order attacks in the probing model. The implementation has been benchmarked on an `ARM-Cortex-M4` processor and incurs a total overhead of 25% on the entire ML-KEM768 decapsulation algorithm, compared with 51% overhead for the defeated *bitwise-fine-shuffling* of [23].

Keywords: ML-KEM · Shuffling · Side-channel Attack · Number Theoretic Transform · Countermeasures · Lattice-based cryptography

1 Introduction and related work

Physical attacks pose a serious threat to cryptographic implementations. They can be divided into fault-injection attacks and side-channel attacks. Introduced by Paul Kocher in [15], side-channel attacks rely on physical measurements leaked by hardware circuits such as power consumption [17], electromagnetic

emanation [16], execution time [5] or cache access time [6], to extract information about sensitive values. The most widely adopted countermeasure is masking. The idea is to split the secret variable into random multiple shares and do the computations over these shares instead to break the correlation between the secret and the power consumption [4]. Masking offers an exponential security growth with the masking order d (the number of traces required for a successful attack grows as $\mathcal{O}(N^d)$, where N depends on the noise level). Notwithstanding, it is important to combine it with shuffling. The idea of the shuffling is to randomise the execution order of the sensitive operations in order to make it harder for the attacker to identify the power consumption related to the targeted operation on a trace. This helps prevent highly sophisticated attacks such as Soft-Analytical-Side-Channel-Attacks (SASCA) [21,22]. These attacks exploit the algebraic structure of masked implementations by leveraging knowledge of the algorithm's data flow. For statistically advanced attacks rely on the hypothesis that traces are well aligned. The effort of traces alignment due to the shuffling scales exponentially with the order of masking used. However, it is important to securely implement the shuffling to achieve the desired security level. While combining masking and shuffling provides strong theoretical security, recent works [14,3] demonstrate that attackers can break each countermeasure separately, significantly reducing the attack complexity. For instance, Jendal *et al.* [14] use fault injection to bypass shuffling before attacking the masking with deep learning analysis, while Backlund *et al.* [3] recover the shuffling permutation via side-channel observations to realign traces before attacking the masking. The present work addresses this attack path by proposing a shuffling implementation where the permutation itself is protected through masking. Furthermore, piling up countermeasures comes with a significant cost. Hence, it is paramount to propose robust and fast shuffling implementations especially for embedded devices. In this context, previous research tried to answer the following challenges in different ways:

Reduce the space of permutations. The polynomials involved in ML-KEM and ML-DSA have a size of 256. This means that the shuffled polynomials can be executed in 256! possible orders, which is astronomically beyond the desired security requirement. Therefore, Ravi *et al.* [23] proposed to reduce the space of permutations to gain in efficiency by introducing three shuffling variants in the context of ML-KEM and ML-DSA with each one of them offering a different trade-off between security and performance: *coarse shuffling*, *coarse-in-group shuffling* and *fine-shuffling*.

Improving the quality of the Random Number Generator (RNG). The shuffling uses random generators to generate the permutations. Sampling uniformly at random from such very large spaces in an unbiased way can be challenging in practice. Ziljstra *et al.* [27] use Linear-Feedback-Shift-Register (LFSR). Ravi [23] proposes to use the well-established FY algorithm. While Park [18] insists on the necessity of abandoning cheap solutions such as the biased modular reduction or low entropy algorithms such as cyclic shifts. Moreover, in

[19], Park *et al.* provide a fast statistical verification method to identify the bias in RNGs.

Design hardware accelerators for the shuffling operation. The purpose of hardware acceleration is to maximise parallelisation and data reuse. Ziljstra *et al.* [27] proposed to use *Benes networks* instead of FY algorithm to increase data parallelisation. Xu *et al.* in [26] introduced a FY implementation tailored for ML-KEM. Indeed, FY has an intrinstic problem which is to generate random indices with dynamic range. This is particularly a bottleneck in the hardware. Instead, they used a fixed-width RNG to generate random numbers and modulo reductions to map these to the current valid range.

Security evaluation of shuffled operations. Despite the theoretical proofs on the shuffling, shuffled implementations are still vulnerable. In [11], Hermelink *et al.* shows that under an expanded attacker model, it is possible to break the *fine-shuffling* and *coarse-in-group* shuffling variants with using SASCA. However, the attacks were limited to simulation. In [3], Backlund *et al.* break the *coarse-shuffling* in ML-KEM by targeting the computation of the swap index inside the FY implementation first to descramble the traces, then perform a classical SASCA attack on unshuffled traces. Goy *et al.* in [10] performed a similar work on HQC and concludes that the fine-shuffling is inherently weak in the case of HQC.

1.1 Contributions

The contributions of this article are the following:

1. **Security analysis of *bitwise-fine-shuffling*.** In [11], Hermerlink *et al.* carried out the attack using SASCA focusing on the random bit used to load the data. The attack was only carried out on simulation and trained the system to directly learn on the masked and shuffled traces. In this work, we attack at a different angle. We perform a template attack by aggregating the leakage from all the bitwise `AND` loop, and validate the attack on a real device (`STM32F303` with electromagnetic measurements) to confirm our theoretical analysis. Our attack strategy differs from [11] in that we first recover the shuffling permutation to realign the traces, rather than attacking the masked and shuffled traces simultaneously. This two-stage approach significantly reduces the training complexity.
2. **`cswap` gadget.** We introduce a new first-order secure conditional swapping gadget (`cswap`) that operates on arithmetic shares of the control bit. This gadget can be used to protect fine-shuffling in ML-KEM and is applicable to other cryptographic contexts requiring masked conditional operations.
3. ***Masked fine-shuffling* implementation.** We introduce an original *masked fine-shuffling* implementation using the `cswap` gadget and we prove it secure against first-order attacks in the probing model. The implementation is secure against the attack we performed on the *bitwise fine shuffling* implementation and also against the attack described in [11]. We bench-marked the solution on an embedded device and quantified the overhead compared with

the unprotected version of ML-KEM768. The implementation we proposed is more secure and faster than the one from the state-of-the art.
Note that we mask the permutation used in the shuffling, not the data being shuffled. Our method is however fully compatible with such data masking, a technique extensively discussed in the literature. We focus on masking the permutation because it is the specific novel part and we want to specifically measure the associated overhead.

Outline. The rest of the paper is organised as follows: Section 2 describes the NTT operation in the context of ML-KEM and introduces the different shuffling variants, section 3 introduces the attack on the *bitwise-fine shuffling* variant. In section 4, we introduce our new masked shuffling method and section 5 concludes the article.

2 NTT operation and its different shuffling variants

Among the lattice-based algorithms, there is a family of algorithms whose security relies on the hardness of Module-Learning-with-Error (MLWE) and Module-Short-Integer-Solution (MSIS) over lattices, such as ML-KEM and ML-DSA. The public, private key and shared keys are represented as polynomials in $\mathcal{R}_q$ or vectors of polynomials in $\mathcal{R}_q^k$. The key generation, encryption and decryption operations involve performing polynomial multiplications over these keys. For that purpose we use the NTT because it has a low algorithmic complexity of $O\left(n \log n\right)$ and offers a significant speedup compared with the schoolbook multiplication [1]. The leakage in the NTT has lead to full secret key recovery in ML-KEM as shown in previous works [21,20]. Hence, it is relevant to mask and shuffle this operation.

2.1 Shuffling as a countermeasure

Performing power analysis requires attacking operations that depend on some parts of the secret key. Therefore, it is important to know the exact moment where these operations happen to capture the power consumption at the right time. The idea of the shuffling is to break the correspondance between the time-points and the operations that are being executed. This goes by randomising the order of execution of the operations. The order is randomly generated at each execution. For example, the AES involves the performing `SBOX` operation on each of the 16 bytes of the key. So instead of applying the `SBOX` in a predictable order,

$$\texttt{SBOX}[state[0]],\ \texttt{SBOX}[state[1],\ \ldots,\ \texttt{SBOX}[state[15]]]$$

one can apply the states in a random order.

$$\texttt{SBOX}[state[7]],\ \texttt{SBOX}[state[2],\ \ldots,\ \texttt{SBOX}[state[3]]]$$

Notice that there are two conditions for the shuffling to be effective: the shuffled operations shall be independent to preserve the correctness of the algorithm.

Moreover, they should have similar power consumption and execution time, because otherwise, the order of the execution can be recovered simply using Simple Power Analysis (SPA) *i.e.* the naked eye.

2.2 Description of the NTT

To deeply understand the mechanisms of the NTT we refer the reader to [25]. In this section, we briefly recall the algorithm of the NTT (cf. Alg. 1) and focus on the different shuffling strategies.

The algorithm operates over a polynomial represented as an array of 256 coefficients and outputs its NTT representation in a bit-reversed order. The algorithm follows a hierarchichal structure with three nested loops:

1. Stage loop (Line 2): Iterates over $\log_2(256) = 8$ stages with len halving the size of the butterfly blocks at each iteration. We call a block of butterflies at a specific stage, the set of butterfly operations that share the same *twiddle factor* ζ^k.
2. Block loop (Line 3): For each stage, we process $\frac{128}{len}$ independent blocks of size $2 \cdot len$.
3. Butterfly loop (Line 5): Within each block, we perform len butterfly operations on coefficients $p[j]$ and $p[j + len]$. A butterfly operation takes two inputs, multiplies one by a root-of-unity called a *twiddle factor* using Montgomery reduction and outputs their modular sum and difference efficiently mixing the pair as a building block of the transform.

Algorithm 1 In-Place Number Theoretic Transform NTT

Require: Array $p[0 \dots 255]$
Ensure: p contains the NTT in bit-reversed order

```
1:  k ← 1
2:  for len ← 128 downto 2 by halving do                ▷ NTT stages
3:      for start ← 0 to 255 step 2 · len do            ▷ Block loop
4:          k ← k + 1                                   ▷ Load twiddle factor
5:          for j ← start to start + len − 1 do         ▷ Butterflies inside block
6:              t ← MontgomeryMul(ζ^k, p[j + len])
7:              p[j + len] ← p[j] − t
8:              p[j] ← p[j] + t
9:          end for
10:     end for
11: end for
```

So, we identify the following independent operations:

– In each stage, the butterfly operations rely on the input from the previous stages, therefore, butterfly operations belonging to different stages cannot be shuffled (cf. Fig. 1).

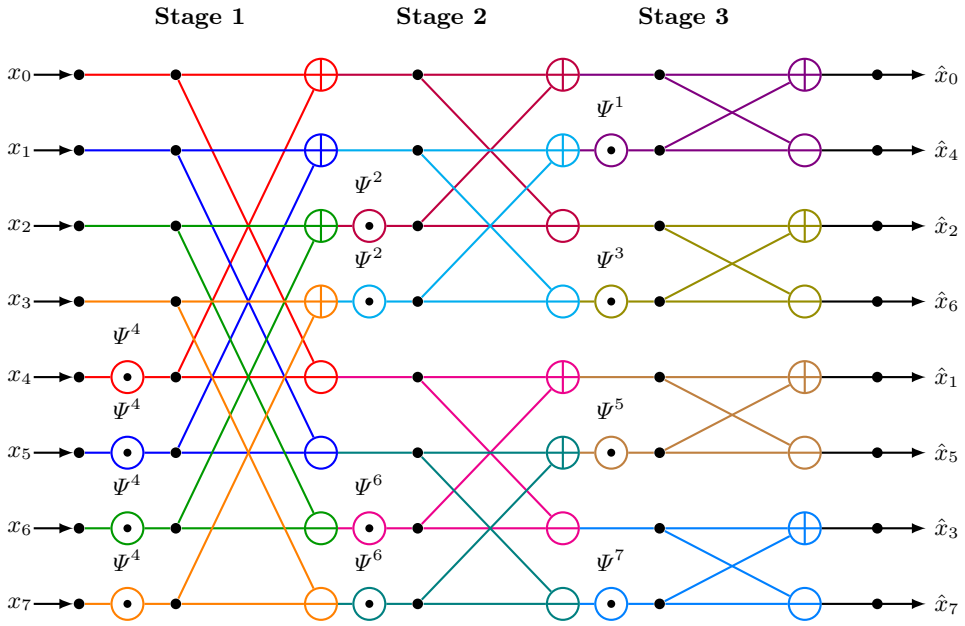


Fig. 1: Data flow graphs of NTT operation following Cooley-Tukey implementation for $n = 3$ stages

– Within the same stage, the blocks are completely independent, therefore, the execution can happen at any order.
– Within the same block, all the butterfly operations are independent from one another, therefore they can be shuffled.

2.3 Shuffling variants of the NTT

In light of the data-dependency analysis in the previous section, three variants have been proposed in [23] and each one of them offers a different level of trade-off between security and optimisation.

1. *coarse NTT shuffling*: The idea is to permute all the butterfly operations regardless of block they belong to. This is slow partly because it carries the overhead of having to load each time the corresponding *twiddle factor* ζ^k.
2. *coarse group NTT shuffling*: This variant overcomes the previous limitation. The idea is to shuffle the operations only within the same the block. Once a block is chosen, we execute all the butterflies in a shuffled order before moving to the next one. This way, we can get rid of the burden of loading the *twiddle factor* each time offering hence a speed-up.
3. *basic fine shuffling*: The butterflies execute in a predictable order. However, the order of storage and loading of the operands inside the butterfly operation is randomised. It is the fastest because it involves permuting only two

elements. It has a smaller space of permutations, albeit sufficient from an entropy point of view. This technique has already been used in elliptic curves in the previous works [12].

Alg. 2 describes how *fine-shuffling* unfolds. The idea is to protect the `load` and `store` operations that involve manipulating $p[j]$ and $p[j + len]$ in Alg. 2 by randomising the order at which they are loaded and stored to op_1 and op_2 and from out_1 and out_2 using a conditional swap. The decision to swap or not is based on random bit sampled at each execution. So, even if an attacker manages to recover op_1 or op_2 using a template attack for example, they don't know whether the recovered op_1 values corresponds to $p[j]$ or $p[j + len]$. The same reasoning applies to the storage using out_1 and out_2. However, this *basic* implementation suffers from an obvious flaw. Using m to decide whether to swap or not is substantially leaky because the difference of between `0` and `0xFFFF` is highly distinguishable even in very noisy environments [23].

Algorithm 2 Basic Fine Shuffling

Require: Array $p[0 \dots 255]$
Ensure: p contains the NTT in bit-reversed order

1: $k \leftarrow 1$
2: **for** $len \leftarrow 128$ **downto** 2 **by** halving **do** ▷ NTT stages
3: **for** $start \leftarrow 0$ **to** 255 **step** $2 \cdot len$ **do** ▷ Block loop
4: $k \leftarrow k + 1$ ▷ Load twiddle factor
5: **for** $j \leftarrow start$ **to** $start + len - 1$ **do** ▷ Butterflies inside block
6: $r \xleftarrow{\$} \{0, 1\}$ ▷ $r \in \{0, 1\}$
7: $m \leftarrow -r$ ▷ If $r = 0$, `m = 0x0000`, else `m = 0xFFFF`
8: $op_1 \leftarrow p[j + r \cdot len]$ ▷ Secret-dependent memory access
9: $op_2 \leftarrow p[j + (1 - r) \cdot len]$ ▷ Secret-dependent memory access
10: $temp \leftarrow (op_1 \oplus op_2) \wedge m$
11: $op_1 \leftarrow op_1 \oplus temp$
12: $op_2 \leftarrow op_2 \oplus temp$
13: $t \leftarrow \text{MontgomeryMul}(op_2, \zeta^k)$
14: $out_1 \leftarrow op_1 + t$
15: $out_2 \leftarrow op_1 - t$
16: $r \xleftarrow{\$} \{0, 1\}$ ▷ $r \in \{0, 1\}$
17: $m \leftarrow -r$ ▷ If $r = 0$, `m = 0x0000`, else `m = 0xFFFF`
18: $temp \leftarrow (out_1 \oplus out_2) \wedge m$
19: $out_1 \leftarrow out_1 \oplus temp$
20: $out_2 \leftarrow out_2 \oplus temp$
21: $p[j + r \cdot len] \leftarrow out_1$ ▷ Secret-dependent memory access
22: $p[j + (1 - r) \cdot len] \leftarrow out_2$ ▷ Secret-dependent memory access
23: **end for**
24: **end for**
25: **end for**

3 Permutation recovery of *bitwise fine-shuffling*

3.1 Description of the bitwise fine-shuffling

To harden the implementation, Ravi introduced the *bitwise fine-shuffling*. The idea is to apply this mask m bit by bit instead as described in Alg. 3. This suppresses the leakage coming from m because it is much more difficult to distinguish `0` from `1` than `0` from `0xFFFF`. However, this solution comes with an overhead of 59% over the entire algorithm. Furthemore, this implementation suffers from two major flaws: first, loading the operations involves handling secret-dependent memory accesses which can be exploited using the *mixing priors* technique mentioned by [11]. Another major flaw, is despite it being more difficult to distinguish `0` from `1` than `0` from `0xFFFF` on a single trace, the `AND` operation is repeated 16 times (32 in the case of ML-DSA), which means that an attacker can aggregate the leakage from all the leakage points and find the value of r with a very high level of confidence thus toppling the shuffling altogether.

3.2 Attack on simulation

In this section, we explain how we can recover the permutation of the *bitwise fine shuffling*. We realised the attack first in simulation. The idea is to observe the moment when the control bit r is loaded before the `AND` operation in Line 11 is executed (cf. Alg. 3). We assume as usual that the leakage follows the Hamming weight model for a variable x [17]:

$$l(x) = \omega(x) + N^{(t)}$$

This means that the leakage is composed of a deterministic part (the Hamming weight of the input x of the algorithm) and a random noise. We assume classically as well that the deterministic part and the noise are independent from one another, that the different noise sources are independent and that the noise follows a Gaussian distribution $N \leftarrow \mathcal{N}(0, \sigma^2)$.

Let L_i the random variable that tracks the leakage obtained at time i where r is manipulated and let R the random variable that tracks the value of the control bit r.

$$L_i = \omega(R) + N^{(i)}$$

Then,

$$L_i \mid (R = r) \leftarrow \mathcal{N}(\omega(r), \sigma^2)$$

Let $L = (L_1, \ldots, L_{16})$ the random variable that tracks the combined leakage. We use the maximum likelihood rule to guess the control bit.

$$r_{guess} = \arg\max_{r \in \{0,1\}} (P(R = r \mid L = l))$$

Algorithm 3 Bitwise fine-shuffling

Require: Array $p[0 \dots 255]$
Ensure: p contains the NTT in bit-reversed order

$k \leftarrow 1$
for $len \leftarrow 128$ **downto** 2 **by** halving **do** ▷ NTT stages
  for $start \leftarrow 0$ **to** 255 **step** $2 \cdot len$ **do** ▷ Block loop
    $k \leftarrow k + 1$ ▷ Load twiddle factor
    for $j \leftarrow start$ **to** $start + len - 1$ **do** ▷ Butterflies inside block
      $r \xleftarrow{\$} \{0, 1\}$ ▷ $r \in \{0, 1\}$
      $op_1 \leftarrow p[j + r \cdot len]$
      $op_2 \leftarrow p[j + (1 - r) \cdot len]$
      $temp \leftarrow (op_1 \oplus op_2)$
      for $i \leftarrow 0$ **to** 15 **do**
        $temp \leftarrow temp \wedge (r \ll i)$ ▷ r bitwise leaky iterations
      end for
      $op_1 \leftarrow op_1 \oplus temp$
      $op_2 \leftarrow op_2 \oplus temp$
      $t \leftarrow \textsc{MontgomeryMul}(op_2, \zeta^k)$
      $out_1 \leftarrow op_1 + t$
      $out_2 \leftarrow op_1 - t$
      $r \xleftarrow{\$} \{0, 1\}$ ▷ $r \in \{0, 1\}$
      $temp \leftarrow (out_1 \oplus out_2)$
      for $i \leftarrow 0$ **to** 15 **do**
        $temp \leftarrow temp \wedge (r \ll i)$ ▷ r bitwise leaky iterations
      end for
      $out_1 \leftarrow out_1 \oplus temp$
      $out_2 \leftarrow out_2 \oplus temp$
      $p[j + r \cdot len] \leftarrow out_1$
      $p[j + (1 - r) \cdot len] \leftarrow out_2$
    end for
  end for
end for

Using Bayes' theorem, we have:

$$P(R = r \mid L = l) = \frac{P(L = l \mid R = r)P(R = r)}{P(L)}$$

Since R follows the uniform distribution, $P(R = r) = \frac{1}{2}$.

$$P(R = r \mid L = l) = \frac{P(L = l \mid R = r)}{2P(L)}$$

Therefore, maximising $P(R = r \mid L = l)$ for r is equivalent to maximising $P(L = l \mid R = r)$ for r. We focus on maximising $P(L = l \mid R = r)$ instead because is it easier to compute. In fact,

$$P(L = (l_1, \ldots, l_{16}) \mid R = r) = \prod_{i=1}^{16} P(L_i = l_i \mid R = r)$$

$$P(L \mid R = r) = \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^{16} \exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{16}\left(l_i - \omega(r)\right)^2\right)$$

$$r_{guess} = \arg\max_{r\in\{0,1\}}(P(L \mid R = r))$$

To determine the success rate of the attack, we need to compare $P(L \mid R = 1)$ and $P(L \mid R = 0)$. Comparing $P(L \mid R = 1)$ and $P(L \mid R = 0)$ is equivalent to comparing $\sum_{i=1}^{16}\left(l_i - \omega(0)\right)^2$ and $\sum_{i=1}^{16}\left(l_i - \omega(1)\right)^2$.

$$P(L \mid R = 0) \geq P(L \mid R = 1) \iff \sum_{i=1}^{16}\left(l_i - 0\right)^2 \leq \sum_{i=1}^{16}\left(l_i - 1\right)^2$$

$$\iff \sum_{i=1}^{16} l_i^2 \leq \sum_{i=1}^{16}\left(l_i^2 - 2l_i + 1\right) \iff \frac{\sum_{i=1}^{16} l_i}{16} \leq \frac{1}{2}$$

Let's denote by $\bar{L} = \frac{\sum_{i=1}^{16} L_i}{16}$ the average of the leakage obtained from all the 16 leakage points in a single trace. So, we simply check the condition $\bar{L} \leq \frac{1}{2}$ and make a decision based on that.

Failure occurs when one of the two events happen:

$$\left((\bar{L} \geq \frac{1}{2}) \cap (R = 0)\right) \cup \left((\bar{L} \leq \frac{1}{2}) \cap (R = 1)\right)$$

This is equivalent to the rule:

$$r_{guess} = \arg\min_{r\in\{0,1\}} \mid \bar{l} - \omega(r) \mid$$

We have:

$$\bar{L} = \omega(R) + \frac{\sum_{i=1}^{16} N^{(i)}}{16}$$

The probability of failure is

$$P_{fail} = P\Big(\bar{L} > \frac{1}{2} \mid R = 0\Big) + P\Big(\bar{L} \leq \frac{1}{2} \mid R = 1\Big)$$

$$P_{fail} = P\Big(\frac{\sum_{i=1}^{16} N^{(i)}}{16} > \frac{1}{2}\Big) + P\Big(\frac{\sum_{i=1}^{16} N^{(i)}}{16} \leq -\frac{1}{2}\Big)$$

$$P_{fail} = 2 \cdot P\Big(\frac{\sum_{i=1}^{16} N^{(i)}}{16} \geq \frac{1}{2}\Big)$$

Since $N \leftarrow \mathcal{N}(0, \sigma^2)$ then, $\frac{\sum_{i=1}^{16} N^{(i)}}{16} \leftarrow \mathcal{N}\Big(0, \frac{\sigma^2}{16}\Big)$
Hence,

$$P_{fail} = 8 \int_{\frac{1}{2}}^{+\infty} \frac{e^{\frac{-8x^2}{\sigma^2}}}{\sigma\sqrt{2\pi}} dx$$

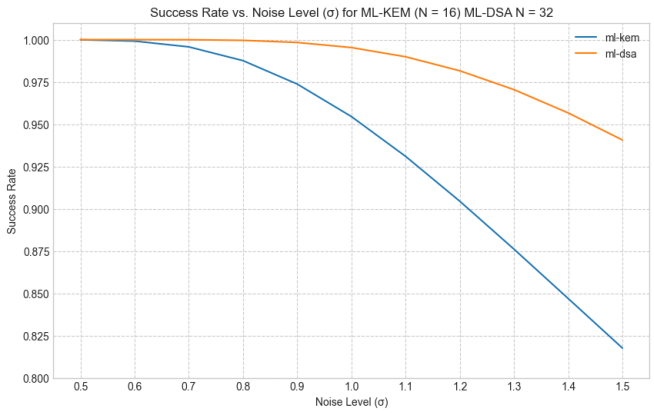


Fig. 2: Success rate of the attack as a function of noise-level for ML-KEM and ML-DSA

Fig. 2 shows the success rate for the attack. It stays above $p = 99.99\%$ for low noise values (up to $\sigma = 0.5$ for ML-KEM and $\sigma = 0.8$ for ML-DSA). In practice, this means that the chances to unscramble the entire NTT computation of all the 896 butterflies are $p^{896} = 91.43\%$.

Remark 1.
While we have implemented the attack on ML-KEM, it also extends naturally to ML-DSA. We simply need to replace 16 by 32 in the formulas.

Remark 2.
In ML-KEM, since $q = 3\,329$, the elements have a length of 12 bits. Therefore,

there is no need to iterate over 16 bits in the first place. This decreases the performance without providing any additional security. Similarly, in ML-DSA, the number of iterations can be brought down to 23 instead of 32 for the same reasons.

3.3 Attack on real implementation

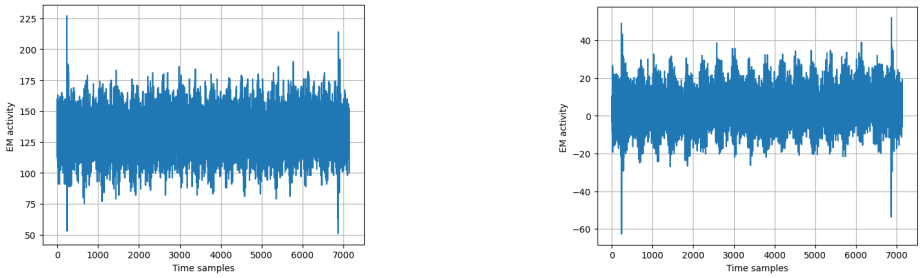
3.3.1 Setup Description In order to validate the result, we performed the attack on a `STM32F303` target. The setup is comprised of:

- `STM32F303` target equipped with `ARM-Cortex-M4` that runs at a frequency of 72 `MHZ`. (≈ 20\$)
- `Chipwhisperer CW1173` that flashes the target. (≈ 300\$)
- Oscilloscope `KeySight MSOX6004A` that captures the electromagnetic emanation from the target. ($\approx 30,000$\$)
- Probe `YWBL-WH`to measure the ElectroMagnetic (EM) signal (10\$) .
- Sampling rate $= 20Ga/s$.
- We put the `TRIGGER_HIGH` at the beginning of the portion of code executing the fine-shuffling and `TRIGGER_LOW` at the end of the fine shuffling (16 loop operation).
- The EM probe was positioned near the decoupling capacitor of the microcontroller. The optimal position was determined empirically by testing multiple spatial positions (x, y, z coordinates) until observing a repeating pattern corresponding to the 16 loop iterations of the *bitwise-fine-shuffling* (lines 10-12 of Alg. 3).
- To increase the quality of the traces, we reduced the jitter using the *window average sliding* method with a filter size of 3 as described in [8].

3.3.2 Template attack While the Hamming weight model is a functional representation of the leakage, it fails to describe how this model scales into electromagnetic emanation which is concretely the physical measurement captured by the attacker. This correspondence between theoretical leakage and the electromagnetic emanation is board specific and that is why we need a training phase. It allows to estimate the statistical distribution of the leakage necessary to build the templates. We refer the reader to [7] for further details.

3.3.3 Attack scenario We performed the attack on the *bitwise-shuffling* implementations from [23] [3]. The code has been compiled with the compiler `arm-none-eabi-gcc` with the compilation options `-O3 -Wall -mcpu=cortex-m4`. We suppose that the attacker has access to a clone device. For the training phase, we used 1000 traces per class. Using the *bitwise shuffling*, the input of MontgomeryMul is shuffled. The idea is to recover the permutation to realign the traces and perform a traditional attack on realigned traces as performed by

[3] code is available on https://github.com/PRASANNA-RAVI/Configurable_SCA_Countermeasures_for_NTT



(a) raw trace (b) filtered trace

Fig. 3: Jitter reduction

Pessl *et al.* in [20]. A trace corresponds to an execution of the NTT function of ML-KEM where we target the control bit r during the 16 iterations of the AND operations (Lines 15 and 21 of Alg. 3). Since the shuffling is unique per execution, the attack traces is comprised of a single-trace. In this experiment, we only show how to break the shuffling, we do not go to the extent of performing the full secret key recovery attack described by Pessl.

3.3.4 Traces filtering The jitter (slight timing misalignments between traces) is a very common phenomenon that degrades the quality of the traces in side-channel analysis. It is due to the imperfection in the measurements instruments or clock instability. To reduce its effect, we used *window average sliding* method described in [8] with a filter size of 3. Fig. (a) shows a raw trace and (b) shows the same trace after applying the *sliding window method*. We can notice that the 16 peaks related to the activity are more visible. This is important to build clean templates for the training phase to increase the success rate.

3.3.5 Points of Interest In order to find the exact moments where the random bit leaks, we use the *sum of differences method* described in [24]. The idea is to compute the absolute value of the average trace of each class.

Fig. 4 shows the points of interests. We can see that we observe 16 peaks which correspond to the loop. However, in practice, we take 16×3 points of interest. In theory, 16 should suffice. Nevertheless, although the variable is only manipulated once, the leakage is spread across multiple samples around the same timepoint.

3.3.6 Results To determine the success rate of the attack, we generate a single execution with random value of r and we compute $P(L = l \mid R = r)$ for both hypothesis based on its corresponding trace. We repeat the experiment a 100 times and plot the results (cf. Fig. 5).

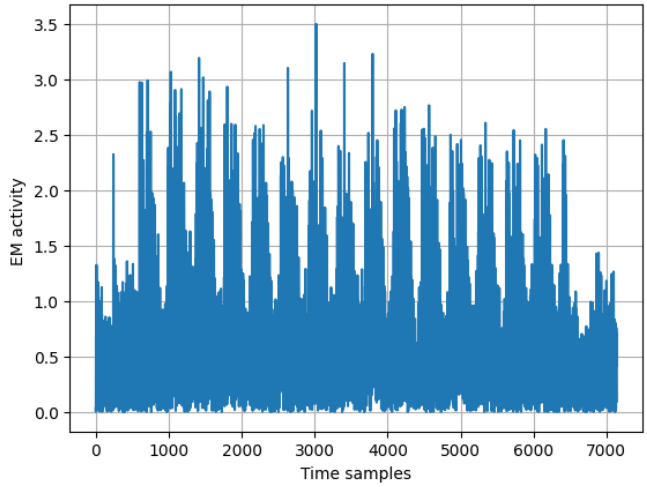


Fig. 4: Points of interests using the method of sums of differences

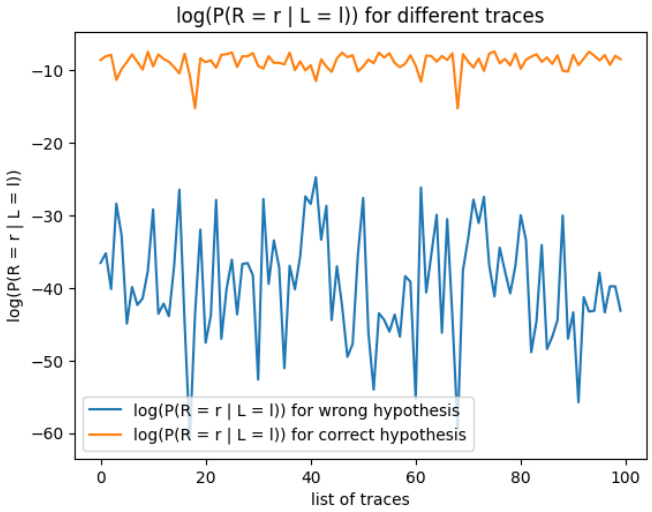


Fig. 5: Comparison of estimated Likelihoods of the correct hypothesis (orange) and false hypothesis (blue) on the control bit r

For all 100 experiments, the correct hypothesis achieves the highest likelihood, we can see that the correct hypothesis has always the highest likelihood, which means that we are able to recover the swap with almost certainty. We did a similar work for the *basic-fine shuffling* implementation and we obtain the same results. Afterall, Both implementations were defeated with the same num-

ber of training traces. Therefore, it is irrelevant to use the *bitwise-fine shuffling* in practice because it incurs a high overhead without increasing the security significantly.

4 Masked fine-shuffling implementation

4.1 Purpose of masked fine-shuffling

To sum up, section 3 and the works of [11] and [3] show that no variant of the shuffling is safe. Therefore, it is relevant to propose hardened/masked versions of these shuffling variants. The fine-shuffling is the lightweight variant compared with *coarse* and *coarse-in-group* as noted by Ravi [23]. In this section, we focus on masking that first variant, prove its security in the probing model and benchmark its performance against existing implementations.

4.2 Description of a new masked fine-shuffling implementation

We propose a masked implementation of the *fine-shuffling* that fixes the two vulnerabilities mentioned:

- The leakage of the secret dependent memory accesses exploited in [11].
- The leakage related to the bitwise application of the control bit r exploited in our attack.

4.2.1 Protecting r-dependent memory accesses. To tackle the first vulnerability, we sample the control bit already in a masked way, by sampling each share $r_1^{(b)}$ and $r_2^{(b)}$ independently.The control bit $r = r_1^{(b)} \oplus r_2^{(b)}$ is never explicitly calculated. Then, we replace all the memory accesses manipulating the control bit r (cf. lines 8, 9, 21, 22 in Alg. 2) with memory accesses manipulating only boolean shares of it instead (cf. lines 7, 8, 17-20 of Alg. 5).

4.2.2 Protecting the mask m using `cswap`. To tackle the second vulnerability, we replace the lines 10-12 and lines 18-20 of Alg. 2 corresponding to the application of the mask $m = -r$ by our `cswap` gadget. This gadget applies a boolean sharing/splitting to m and harnesses the well-known `B2A` gadget for masking conversion.

`B2A` gadget. Cryptographic implementations mix in general linear and non-linear operations. We use arithmetic masking for linear operations and boolean masking for non-linear operations. Moving from one domain to the other thus requires either recomposing the shares to generate new shares corresponding to the new domain, but this is obviously not secure because it exposes the secret. The other solution is to securely convert the shares from arithmetic to boolean or the other way around. This is the purpose of the gadgets `A2B` and `B2A` designed by Goubin in [9].

`cswap` gadget.
The `cswap` allows to apply the mask m over boolean shares. The full description of the algorithm can be found in Alg.4.

The algorithm receives boolean sharings of r and converts them to arithmetic sharings using the `B2A` gadgets. Then, samples at random the first boolean share of m, labeled m_1. Then, blindly computes the second boolean share m_2 such that $m = m_1 \oplus m_2$ and $m = 0$ if $r = 0$ and $m = 2^{16} - 1$ otherwise. Then each share is applied individually as logical-`AND` mask to op_1 and op_2.

Algorithm 4 `cswap` gadget

Require: op_1, op_2, $r_1^{(b)}$, $r_2^{(b)}$ such that $r = r_1^{(b)} \oplus r_2^{(b)}$
Ensure: op_1 and op_2 are swapped iff $r = 1$
1: $(r_1^{(a)}, r_2^{(a)}) \leftarrow \texttt{B2A}(r_1^{(b)}, r_2^{(b)})$ ▷ $r_1^{(a)} + r_2^{(a)} = r \mod 2^{16}$
2: $m_1 \xleftarrow{\$} \mathbb{F}_{2^{16}}$
3: $temp \leftarrow 2^{16} - 1 - 2 \cdot m_1$
4: $m_2 \leftarrow temp \cdot r_1^{(a)} + m_1 + temp \cdot r_2^{(a)}$ ▷ $m_1 \oplus m_2 = 0$ if r = 0, -1 otherwise
5: $op1m1 \leftarrow op_1 \wedge m_1$
6: $op2m1 \leftarrow op_2 \wedge m_1$
7: $op1m2 \leftarrow op_1 \wedge m_2$
8: $op2m2 \leftarrow op_2 \wedge m_2$
9: $op12m1 \leftarrow op1m1 \oplus op2m1$
10: $op12m2 \leftarrow op1m2 \oplus op2m2$
11: $op_1 \leftarrow op_1 \oplus op12m1 \oplus op12m2$
12: $op_2 \leftarrow op_2 \oplus op12m1 \oplus op12m2$

4.3 Correctness

To show the correctness of the implementation, we need to show the correctness at each of the following steps that depend on the bit control: loading step, `cswap` gadget and storage step.

Correctness of `cswap` step.
First, we need to introduce the following lemma.

We use the usual two's complement representation to identify $\{0,1\}^n$ with $\mathbb{Z}/2^n\mathbb{Z}$ with the bijection

$$\varphi : \{0,1\}^n \longrightarrow \mathbb{Z}/2^n\mathbb{Z}$$

defined by

$$\varphi(x_{n-1}, \ldots, x_0) = -x_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i \pmod{2^n}.$$

In particular,

$$\varphi(1, \ldots, 1) = -1 = 2^n - 1 \pmod{2^n}.$$

Algorithm 5 first-order masking of shuffling operation in NTT

Require: Array $p[0 \dots 255]$
Ensure: p contains the NTT in bit-reversed order

1: $k \leftarrow 1$
2: **for** $len \leftarrow 128$ **downto** 2 **by** halving **do** ▷ NTT stages
3: **for** $start \leftarrow 0$ **to** 255 **step** $2 \cdot len$ **do** ▷ Block loop
4: $k \leftarrow k + 1$ ▷ Load twiddle factor
5: **for** $j \leftarrow start$ **to** $start + len - 1$ **do** ▷ Butterflies inside block
6: $r_1^{(b)}, r_2^{(b)} \leftarrow \{0, 1\}$ ▷ $r_1^{(b)} \oplus r_2^{(b)} = r \in \{0, 1\}$
7: $op_1 \leftarrow p[j + r_1^{(b)} \cdot len] \oplus p[j] \oplus p[j + r_2^{(b)} \cdot len]$
8: $op_2 \leftarrow p[j + (1 - r_1^{(b)}) \cdot len] \oplus p[j + len] \oplus p[j + (1 - r_2^{(b)}) \cdot len]$
9: cswap$(op_1, op_2, r_1^{(b)}, r_2^{(b)})$
10: $t \leftarrow \text{MontgomeryMul}(\zeta^k, op_2)$
11: $out_1 \leftarrow op_1 + t$
12: $out_2 \leftarrow op_1 - t$
13: $r_1^{(b)}, r_2^{(b)} \leftarrow \{0, 1\}$
14: cswap$(out_1, out_2, r_1^{(b)}, r_2^{(b)})$
15: $p[j] \leftarrow out_1$
16: $p[j + len] \leftarrow out_2$
17: $p[j + r_1^{(b)} \cdot len] \leftarrow p[j + r_1^{(b)} \cdot len] \oplus out_2$
18: $p[j + (1 - r_1^{(b)}) \cdot len] \leftarrow p[j + (1 - r_1^{(b)}) \cdot len] \oplus out_1$
19: $p[j + r_2^{(b)} \cdot len] \leftarrow p[j + r_2^{(b)} \cdot len] \oplus out_2$
20: $p[j + (1 - r_2^{(b)}) \cdot len] \leftarrow p[j + (1 - r_2^{(b)}) \cdot len] \oplus out_1$
21: **end for**
22: **end for**
23: **end for**

Lemma.

$$\forall x, y \in \{0,1\}^n, x \oplus y = -1 \Leftrightarrow x + y = 2^n - 1 \pmod{2^n}.$$

Proof of Lemma. For all $x, y \in \{0,1\}^n$,

$$\begin{aligned} x \oplus y = -1 &\Leftrightarrow\ x \oplus x \oplus y = -1 \oplus x \\ &\Leftrightarrow\ (x \oplus x) \oplus y = \neg x \\ &\Leftrightarrow\ y = \neg x \\ &\Leftrightarrow\ x + y = \neg x + x \pmod{2^n} \\ &\Leftrightarrow\ x + y = 2^n - 1 \pmod{2^n}. \end{aligned}$$

□

Now let us prove the correctness of our `cswap` gadget.

- The last statements of `cswap`, consist in xor-ing

$$op12m1 \oplus op12m2 = (op_1 \oplus op_2) \wedge (m_1 \oplus m_2)$$

  to op_1 and op_2.
- With the lemma, we have $m_1 \oplus m_2 = 0$ if $r = 0$, and $m_1 \oplus m_2 = -1$ otherwise.
- If $r = 0$, $op12m1 \oplus op12m2 = 0$, so $op_1 \leftarrow op_1$ and $op_2 \leftarrow op_2$. No swap.
- If $r = 1$, $op12m1 \oplus op12m2 = op_1 \oplus op_2$, so $op_1 \leftarrow op_2$ and $op_2 \leftarrow op_1$. The elements are swapped.

Correctness of loading step.
The idea is that if $r = 0$, we `XOR` op_1 and op_2 with the same value, thus keeping it unchanged. Otherwise, we `XOR` with $p[j] \oplus p[j + len]$which has the effect of swapping the values inside op_1 and op_2.

- **Case** $r = 0$:

This implies that $r_1^{(b)} = r_2^{(b)}$ and $1 - r_1^{(b)} = 1 - r_2^{(b)}$, so $op_1 \leftarrow p[j]$ and $op_1 \leftarrow p[j + len]$. So the elements are loaded without swapping as expected.

- **Case** $r = 1$:

This implies that $r_1^{(b)} \neq r_2^{(b)}$ and $1 - r_1^{(b)} \neq 1 - r_2^{(b)}$. So, one element is $p[j]$ and the other one is $p[j+len]$. Without loss of generality, let's assume that $r_1^{(b)} = p[j]$ and $r_2^{(b)} = p[j + len]$. In that case, $op_1 \leftarrow p[j + len]$ and $op_2 \leftarrow p[j]$. Hence, the elements loaded swapped as expected.

Correctness of storage step.
In the storage step, if $r = 0$, *i.e.* $r_1 = r_2$, we `XOR` twice $p[j]$ and $p[j + len]$ by the same value either out_1 or out_2 which keeps the values of $p[j]$ and $p[j + len]$ unchanged. If $r = 1$, *i.e.* $r_1 = r_2$, we `XOR` $p[j]$ and $p[j+len]$ by $out_1 \oplus out_2$ which has the effect of swapping the values inside $p[j]$ and $p[j + len]$.

– **Case** $r_1^{(b)} = r_2^{(b)} = 0$:

$p[j] \leftarrow out_1$
$p[j + len] \leftarrow out_2$
$p[j] \leftarrow p[j] \oplus out_2 = out_1 \oplus out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_1 = out_1 \oplus out_2$
$p[j] \leftarrow p[j] \oplus out_2 = out_1 \oplus out_2 \oplus out_2 = out_1$
$p[j + len] \leftarrow p[j + len] \oplus out_1 = out_1 \oplus out_2 \oplus out_1 = out_2$
So, no swap happens as expected.

– **Case** $r_1^{(b)} = r_2^{(b)} = 1$:

$p[j] \leftarrow out_1$
$p[j + len] \leftarrow out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_2 = 0$
$p[j] \leftarrow p[j] \oplus out_1 = 0$
$p[j + len] \leftarrow p[j + len] \oplus out_2$
$p[j] \leftarrow p[j] \oplus out_1 = out_1$
So, no swap happens as expected.

– **Case** $r_1^{(b)} = 1, r_2^{(b)} = 0$:

$p[j] \leftarrow out_1$
$p[j + len] \leftarrow out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_2 = 0$
$p[j] \leftarrow p[j] \oplus out_1 = 0$
$p[j] \leftarrow p[j] \oplus out_2 = out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_1 = out_1$
So, the swap happens as expected.

– **Case** $r_1^{(b)} = 0, r_2^{(b)} = 1$:

$p[j] \leftarrow out_1$
$p[j + len] \leftarrow out_2$
$p[j] \leftarrow p[j] \oplus out_2 = out_1 \oplus out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_1 = out_1 \oplus out_2$
$p[j + len] \leftarrow p[j + len] \oplus out_2 = out_1 \oplus out_2 \oplus out_2 = out_1$
$p[j] \leftarrow p[j] \oplus out_1 = out_1 \oplus out_2 \oplus out_1 = out_2$
So, the swap happens as expected. This concludes the correctness of the method.

4.4 Proof of security in the probing model

We propose to prove the security under the probing model. In 2003, Ishai *et al.* introduce the probing model [13] to formalize the security of masking schemes against adversaries capable of observing a bounded number of internal wires/intermediate variables in an implementation. In this model, the attacker is able to observe at most t variables. An implementation is said to be t-probe secure if any set of at most t intermediate variables does not yield any information

about the secret. In other words, to prove that an implementation is 1-probe secure, we have to show that for all intermediate variables V are independent of R. Notice that the control bit r is already generated in a masked way, is never recomposed and no intermediate variable depends on both shares of the bit control. This is what makes the implementation robust against first-order attacks.

$$P(R = r \mid V = v) = P(R = r)$$

or equivalently

$$P(V = v \mid R = r) = P(V = v)$$

4.4.1 `cswap` gadget The algorithm has been designed in such a way that no intermediate variable mixes the two shares of a (sensitive) variable that directly depends on r *i.e.* a function of $((r_1^{(b)}, r_2^{(b)})$ or $(r_1^{(a)}, r_2^{(a)})$ or $(m_1, m_2))$ without rerandomisation. Let V_i denote the set of all intermediate variables that appear at statement i of Alg. 4.

- $V_1^{(1)} = r_1^{(b)}$, $V_1^{(2)} = r_2^{(b)}$, $V_1^{(3)} = r_1^{(a)}$, $V_1^{(4)} = r_2^{(a)}$, as well as all internal variables while executing gadget `B2A` are by design independent of R.
- $V_2 = m_1$ is picked uniformly at random, hence independent of R.
- $V_3 = 2^{16} - 1 - 2 \cdot V_2$ only depends on V_2, hence independent of R.
- Let $V_4^{(1)} = V_3.V_1^{(3)}$, $V_4^{(2)} = V_4^{(1)} + V_2$, $V_4^{(3)} = V_3.V_1^{(4)} = m_2$, $V_4^{(4)} = V_4^{(2)} + V_4^{(3)} = m_2$.
  V_3 is unrelated to r or its shares, and $V_1^{(3)}$ formally only depends on $r_1^{(b)}$ therefore independent of R, hence $V_4^{(1)}$ is independent of R.
  V_2 is unrelated to r or its shares, and $V_4^{(1)}$ is independent of R, hence $V_4^{(2)}$ is independent of R.
  V_3 is unrelated to r or its shares, and $V_1^{(4)}$ is independent of R, hence $V_4^{(3)}$ is independent of R.
  $V_4^{(4)} = m_2$ that is distributed as m_1 hence independent of R.
- V_5, V_6, V_7, V_8 are all in the form $op_i \wedge m_j$ with $i, j \in \{1, 2\}$; there are independent of R, as m_j was already proven independent of R, and the definition of op_i is independent of r or its shares.
- V_9, V_{10} are of the form $op_i \oplus m_j$, i.e. never recombine m_1 and m_2; as op_i does not formally depend on r or m, and m_j does not depend on R, V_9 and V_{10} also do not depend on R.
- (V_{11}, V_{12}) compute the output without ever recombining m_1 with m_2 before the very last addition, thus revealing nothing more than the information already contained in the output of the gadget.

Remark 3.
If we want to use the `cswap` gadget in a more general setting, where op_1 and op_2 are jointly sensitive, we have to ensure that beyond the independence from

r, no intermediate variable recombines op_1 and op_2. Our `cswap` gadget is purposely designed so that no such intermediate variable appears until the very last statements. The only update required to make our gadget definition completely free of any such combination consists in introducing an additional random value middle computation of $op12m1$ and $op12m2$ in statements $9, 10$ acting as an intermediate mask as follows:

$$op12m1 \leftarrow op1m1 \oplus m_3 \oplus op2m1$$

$$op12m2 \leftarrow op1m2 \oplus m_3 \oplus op2m2$$

where m_3 is picked at random in $\mathbb{F}_{2^{16}}$. In so doing, no intermediate variable ever recombines op_1 and op_2. The m_3 masks simply cancel out in statements $11, 12$ as $op12m1$ and $op12m2$ are added. Note however that this comes at the price of an additional random value, hence to be avoided if not necessary (as in our case of shuffling the NTT operation).

4.4.2 Masked memory access Let V_i denote the set of all intermediate variables that appear at statement i of Alg. 5.

- $V_6^{(1)} = r_1^{(b)}$, $V_6^{(2)} = r_2^{(b)}$ are generated independently at random, therefore, they are independent of R.
- $V_7^{(1)} = j + r_1^{(b)} \cdot len = j + V_6^{(1)} \cdot len$ only depend on $V_6^{(1)}$, hence independent of R. The same goes for $V_7^{(2)} = j + r_2^{(b)} \cdot len = j + V_6^{(2)} \cdot len$.
- $V_7^{(3)} = p[j + r_1^{(b)}]$ and $V_7^{(4)} = p[j + r_2^{(b)}]$ depend only respectively on $r_1^{(b)}$ and $r_2^{(b)}$, therefore they are independent of R.
- $V_7^{(5)} = p[j + r_1^{(b)} \cdot len] \oplus p[j]$ only depends on $r_1^{(b)}$ therefore is independent of R.
- $V_7^{(6)} = op_1 = p[j + r_1^{(b)} \cdot len] \oplus p[j] \oplus p[j + r_2^{(b)} \cdot len]$. $V_7^{(5)} = op_1$ takes values in $\mathbb{Z}_q$. Let $x \in \mathbb{Z}_q$.

  $$P(op_1 = x \mid R = 0) = P(p[j] = x)$$

  $$P(op_1 = x \mid R = 1) = P(p[j + len] = x)$$

  $p[j]$ and $p[j + len]$ follow the same distribution law and whose value is unknown to the attacker. Therefore,

  $$P(p[j] = x) = P(p[j + len] = x)$$

  $$P(op_1 = x) = P(op_1 = x \mid R = 0) \cdot P(R = 0) + P(op_1 = x \mid R = 1) \cdot P(R = 1) = P(p[j] = x)$$

  $$P(op_1 = x \mid R = r) = P(op_1 = x)$$

  Hence, op_1 and R are independent.
- The proof is analogous for the $V_8^{(i)}$ for $i \in \{1, \ldots, 5\}$
- $V_9^{(1)} = op_1$ and $V_9^{(2)} = op_2$ are independent of R. Proof in the previous section.

- $V_{10} = t$ is independent of R because it only depends on op_2 that is independent of R.
- $V_{11} = op_1 + t$ takes values in $\mathbb{Z}_q$. Let $x \in \mathbb{Z}_q$.

$$P(op_1 + t = x \mid R = 0) = P(p[j] + p[j + len] \cdot \zeta^k = x)$$

$$P(op_1 + t = x \mid R = 1) = P(p[j + len] + p[j] \cdot \zeta^k = x)$$

$p[j]$ and $p[j + len]$ follow the same distribution law and whose value is unknown to the attacker.. Therefore,

$$P(p[j] + p[j + len] \cdot \zeta^k = x) = P(p[j + len] + p[j] \cdot \zeta^k = x))$$

$$P(op_1{+}t = x) = P(op_1{+}t = x \mid R = 0){\cdot}P(R = 0){+}P(op_1{+}t = x \mid R = 1){\cdot}P(R = 1)$$

$$= P(p[j] + p[j + len] \cdot \zeta^k = x)$$

$$P(V_{11} = x \mid R = r) = P(V_{11} = x)$$

Hence, op_1 and R are independent.
- $V_{13}^{(1)} = out_1$ and $V_{13}^{(2)} = out_2$ are independent of R. Proof in the next section.
- The proof is analogous for $V_{12} = op_1 - t$.
- $V_{14} = p[j]$ is independent of R since out_1 is independent of R.
- $V_{15} = p[j + len]$ is independent of R since out_2 is independent of R.
- $V_{16}^{(1)} = p[j + r_1^{(b)} \cdot len] \cdot V_{15}$ is independent of R since it only dependents on $r_1^{(b)}$.
- $V_{16}^{(2)} = p[j + r_1^{(b)} \cdot len] \oplus out_2$ is independent of R because it only depends on $r_1^{(b)}$.
- $V_{17} = p[j + (1 - r_1^{(b)} \cdot len)]$, $V_{18} = p[j + r_2^{(b)} \cdot len]$, $V_{19} = p[j + (1 - r_2^{(b)}) \cdot len]$ can similarly be proven independent of R.

4.5 T-test

To validate experimentally the implementation against first-order leakage, we perform the Test Vector Leakage Assessment (TVLA). The method is based on Welch's test. It measures whether two distinct distributions are statistically distinguishable. The idea is to collect two groups of traces:

- Group 0: traces where $r = 0$ with shares sampled at random.
- Group 1: traces where $r = 1$ with shares sampled at random.

For every sample t in the trace T, we compute Welch's test defined as follows:

$$T(t) = \frac{\mu_0(t) - \mu_1(t)}{\sqrt{\frac{\sigma_0^2(t)}{N_0} + \frac{\sigma_1^2(t)}{N_1}}}$$

where:

- $\mu_0(t)$ and $\mu_1(t)$ mean power at time t over all traces.

– $\sigma_0^2(t)$, $\sigma_1^2(t)$ the variances.
– N_0, N_1 number of traces of each class.

A t-value $|T(t)| > 4.5$ provides with a confidence level of 99.99% that there is a leakage at sample point t. The tests have been executed with $5,000$ traces per group.

Fig. 6 shows the first order leakage, as we can see, t-values remain well below the threshold. This means that no statistically significant leakage was detected.

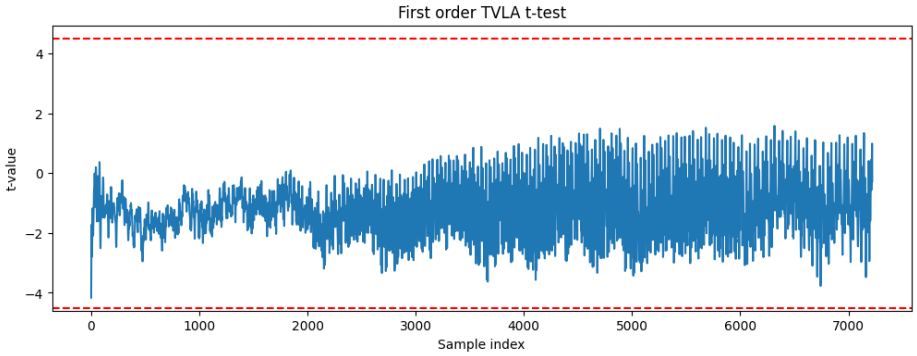


Fig. 6: First-order T-test

4.6 Benchmark

We perform a performance evaluation of the proposed countermeasure when integrated into the reference implementation of ML-KEM [4].

4.6.1 Target Platform and implementation details `STM32F303` does not support hardware accelerated Pseudo-Random Number Generator (PRNG). Hence, the benchmarks were run on the platform `TI CC2642R1-Q1` equipped with an `ARM Cortex-M4F` CPU running at a clock frequency of `48 MHz` and a `SRAM` of 8 KB. For the randomness generation, we use a PRNG using AES hardware accelerator, initialised with a seed sampled by a True Random Number Generator (TRNG). The implementations were compiled using the compiler `TI ARM Clang` with the compilation flags `-O3 -mcpu=cortex-m4 -mfloat -abi=hard -mfpu=fpv4-sp-d16 -mlittle-endian -mthumb`. The AES hardware accelerator consumes $5 - 8$ clock cycles per random byte generated according to the number of bytes requested to be generated. The performance of clock cycles per byte does not scale linearly when we increase the size of the input given. To

[4] The link of the implementation can be found in https://pq-crystals.org/kyber/resources.shtml.

amortise the cost of the PRNG, for both implementations, we made a single call to the AES-Deterministic Random Bit Generator (AES-DRBG) with the necessary number of random bytes to perform an entire single call of NTT/INTT operation and we store the random bytes in a buffer. This comes of course at a cost on the memory. Nevertheess, a trade-off can be found between the security and the memory size according to one's need. Since the buffer is accessed thousands of time during the NTT operations, we have forced the compiler to align the memory address of the buffer to a multiple of 4, and we read each time 32 or 64 bits of data from the buffer. This allows in one hand to reduce the number of memory accesses to the buffer and ensures that each access only costs 1 cycle instead of multiple clock cycles per access. The initial *bitwise-shuffling* implementation of Ravi used a slow hardware TRNG that consumes 54 cycles per byte. Therefore, we connected it to the AES-DRBG as well to provide a fair comparison with our implementation (we obtained an overhead of +281% compared with the original implementation incurred an overhead of +356%).

4.6.2 Performance evaluation of protected NTT/INTT modules Tab. 1 shows the performance of each implementation. We can see overall that our implementation is faster than the one proposed by [23] for both NTT and INTT despite our implementation being protected against first-order attacks. Moreover, we can further optimise the implementation by generating the asynchronously the random bytes thus, completely wipe out the overhead of random generation and reduce the memory impact. To measure the cycle count on `ARM-Cortex-M4` CPU, we read the register `DWT_CYCCNT`. We run the operation $10,000$ times and compute the average. Notice that the overhead of our countermeasure on the INTT (+60%) is significantly lower than the overhead incurred on NTT (+149%) because the cost of the countermeasure is in part diluted in the relatively higher cost of the INTT.

variant	NTT		INTT	
	performance	overhead	performance	overhead
Reference implementation [2]	26, 266	_	44, 443	_
our *bitwise* implementation from [23]	100, 121	281%	111, 079	150%
Present work	65, 551	**149%**	71, 132	**60%**

Table 1: Performance evaluation of NTT/INTT for the different implementations

4.6.3 Performance evaluation of ML-KEM768 Tab. 2 shows the overhead of the countermeasure over the entire decapsulation operation. As expected, we still do better than the *bitwise* implementation of Ravi. It is important to note that the decapsulation operation makes 6 calls to the NTT and 5 calls to INTT. Therefore, the overhead propagates significantly. However, it gets diluted in the entire execution of the algorithm. We end up with a 24% overhead.

variant	ML-KEM implementation	
	performance	overhead
Reference implementation [2]	1,515,900	_
bitwise implementation[23]	2,293,518	51%
Present work	**1,885,946**	**24%**

Table 2: Performance evaluation of decapsulation for the different implementations

5 Conclusion

This work addresses the security of the fine-shuffling in lattice-based cryptography with application to ML-KEM. We first defeated the *bitwise-fine-shuffling* variant proposed by Ravi *et al.* using a new angle of attack by aggregating the leakage from the 16 iterations of the bitwise masking loop. The attack has been performed in simulation and also on a real target `STM32F303` with a very high success rate in practice in realistic noise scenarios. This practical demonstration confirms that the significant performance overhead of *bitwise-fine-shuffling* (+281%) for NTT does not provide security against first-order attacks. For that matter, we introduced an end-to-end masked fine-shuffling implementation. The implementation replaces the secret dependent memory accesses with ones depending only on its shares and uses a new `cswap` gadget to protect the mask for the swapping. Moreover, We have proved this implementation secure against first-order attacks. The implementation happens to be more secure and faster than the *bitwise-fine shuffling* incurring an overhead of only 24% over ML-KEM768.
Limitations and future work. While this study was applied to ML-KEM it can naturally be extended to ML-DSA, and more generally to other contexts such as fully homomorphic encryption and symmetric cryptography. Moreover, it is interesting to unravel how this countermeasure scales to higher-order levels. The proof in the probing model ensures security in the theoretical leakage model, but it does not address the micro-architectural leakage. Furthermore, it does not guarantee the same security level for composability with another gadget. Therefore, it is relevant to evaluate whether the gadget fullfills the composability and provide a hardened the implementation against microarchitectural leakage. Finally, it is important evaluate the overall cost of combining the first-order masking with our fine-shuffling implementation.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Acknowledgments. The authors would like to acknowledge the EU funding received under the FOCAL project (grant No. 101225859). This work was supported in part by French projects ANR-11-LABX-0020-01 “Centre Henri Lebesgue” and ANR-18-EURE-0004 “Cyberschool”. We would like also to thank the reviewers for their relevant feedback that improved the quality of the paper.

References

1. Agarwal, R.C., Burrus, C.S.: Number theoretic transforms to implement fast digital convolution. Proceedings of the IEEE **63**(4), 550–560 (2005)
2. Alkim, E., Avanzi, R., Bos, J., Ducas, L., de la Piedra, A., Pöppelmann, T., Schwabe, P., Stebila, D.: Algorithm specifications and supporting documentation. Version **1**, 1–44
3. Backlund, L., Ngo, K., Gärtner, J., Dubrova, E.: Secret key recovery attack on masked and shuffled implementations of CRYSTALS-kyber and saber. In: International Conference on Applied Cryptography and Network Security. pp. 159–177. Springer (2023)
4. Bos, J.W., Gourjon, M.O., Renes, J., Schneider, T., Vredendaal, C.v.: Masking kyber: First-and higher-order implementations. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(4), 173–214 (2021)
5. Brumley, D., Boneh, D.: Remote timing attacks are practical. Computer Networks **48**(5), 701–716 (2005)
6. Canteaut, A., Lauradoux, C., Seznec, A.: Understanding cache attacks. Ph.D. thesis, INRIA (2006)
7. Chari, S., Rao, J.R., Rohatgi, P.: Template attacks. In: International workshop on cryptographic hardware and embedded systems. pp. 13–28. Springer (2002)
8. Fledel, D., Wool, A.: Sliding-window correlation attacks against encryption devices with an unstable clock. In: International Conference on Selected Areas in Cryptography. pp. 193–215. Springer (2018)
9. Goubin, L.: A sound method for switching between boolean and arithmetic masking. In: International Workshop on Cryptographic Hardware and Embedded Systems. pp. 3–15. Springer (2001)
10. Goy, G., Maillard, J., Gaborit, P., Loiseau, A.: Single trace hqc shared key recovery with sasca. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(2), 64–87 (2024)
11. Hermelink, J., Streit, S., Strieder, E., Thieme, K.: Adapting belief propagation to counter shuffling of ntts. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 60–88 (2023)
12. Hutter, M., Schwabe, P.: Nacl on 8-bit avr microcontrollers. In: International Conference on Cryptology in Africa. pp. 156–172. Springer (2013)
13. Ishai, Y., Sahai, A., Wagner, D.: Private circuits: Securing hardware against probing attacks. In: Annual International Cryptology Conference. pp. 463–481. Springer (2003)
14. Jendral, S., Ngo, K., Wang, R., Dubrova, E.: A single-trace message recovery attack on a masked and shuffled implementation of CRYSTALS-kyber. Cryptology ePrint Archive (2023)
15. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M. (ed.) Advances in Cryptology — CRYPTO' 99. pp. 388–397. Springer Berlin Heidelberg, Berlin, Heidelberg (1999)
16. Longo, J., De Mulder, E., Page, D., Tunstall, M.: SoC it to EM: electromagnetic side-channel attacks on a complex system-on-chip. In: Cryptographic Hardware and Embedded Systems–CHES 2015: 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings 17. pp. 620–640. Springer (2015)
17. Mangard, S., Oswald, E., Popp, T.: Power analysis attacks: Revealing the secrets of smart cards, vol. 31. Springer Science & Business Media (2008)

18. Park, J.Y.: A Study of High-Performance, Low-Cost Software-Hardware Co-Design Implementation for Post Quantum Cryptographic Algorithms. Ph.D. thesis, kyushu University (2025)
19. Park, J.Y., Ju, J.W., Lee, W., Kang, B.G., Kachi, Y., Sakurai, K.: A statistical verification method of random permutations for hiding countermeasure against side-channel attacks. Journal of Information Security and Applications **84**, 103797 (2024)
20. Pessl, P., Primas, R.: More practical single-trace attacks on the number theoretic transform. In: Progress in Cryptology–LATINCRYPT 2019: 6th International Conference on Cryptology and Information Security in Latin America, Santiago de Chile, Chile, October 2–4, 2019, Proceedings 6. pp. 130–149. Springer (2019)
21. Primas, R., Pessl, P., Mangard, S.: Single-trace side-channel attacks on masked lattice-based encryption. In: Cryptographic Hardware and Embedded Systems–CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings. pp. 513–533. Springer (2017)
22. Ravi, P., Chattopadhyay, A., D'Anvers, J.P., Baksi, A.: Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. ACM Transactions on Embedded Computing Systems **23**(2), 1–54 (2024)
23. Ravi, P., Poussier, R., Bhasin, S., Chattopadhyay, A.: On configurable sca countermeasures against single trace attacks for the ntt: A performance evaluation study over kyber and dilithium on the arm cortex-m4. In: International Conference on Security, Privacy, and Applied Cryptography Engineering. pp. 123–146. Springer (2020)
24. Remmerswaal, M.G., Wu, L., Tiran, S., Mentens, N.: Autopoi: automated points of interest selection for side-channel analysis. Journal of Cryptographic Engineering **14**(3), 463–474 (2024)
25. Satriawan, A., Syafalni, I., Mareta, R., Anshori, I., Shalannanda, W., Barra, A.: Conceptual review on number theoretic transform and comprehensive review on its implementations. IEEE Access **11**, 70288–70316 (2023)
26. Xu, D., Wang, K., Tian, J.: A hardware-friendly shuffling countermeasure against side-channel attacks for kyber. IEEE Transactions on Circuits and Systems II: Express Briefs (2025)
27. Zijlstra, T., Bigou, K., Tisserand, A.: Fpga implementation and comparison of protections against scas for rlwe. In: International Conference on Cryptology in India. pp. 535–555. Springer (2019)